



Smart Water Utility Consumption, Billing, and Complaint Management System

IET ASSIGNMENT REPORT

Submitted in the partial fulfilment for the Course of
CSA – 0509 Database Management Systems

to the award of the degree of
BACHELOR OF TECHNOLOGY
IN
COMPUTER SCIENCE & BIOSCIENCE
Submitted by
JEROLD SAMUEL J (192571052)

Under the Supervision of

Dr.R.Kesavan
Professor



SIMATS
ENGINEERING



SIMATS
Saveetha Institute of Medical And Technical Sciences
(Declared as Deemed to be University under Section 3 of UGC Act 1956)

SIMATS ENGINEERING
Saveetha Institute of Medical and Technical Sciences
Chennai-602105

September 2026
SEPTEMBER 2026

**Saveetha Institute of Medical and Technical Sciences
Chennai-602105**

BONAFIDE CERTIFICATE

This is to certify that the Capstone Project entitled “Smart Water Utility Consumption ,Billing and Complaint Manageent Ststem ” has been carried out by JEROLD SAMUEL J (192571052) under the supervision of Dr. R.Kesavan and is submitted in partial fulfilment of the requirements for the current semester of the B.Tech IT program at Saveetha Institute of Medical and Technical Sciences, Chennai.

COURSE COORDINATOR

**Dr.D.Angel
Professor
SIMATS Engineering,
SIMATS, Chennai – 602105.**

COURSE FACULTY

**Dr,R.Kesvan
professor
SIMATS Engineering,
SIMATS, Chennai – 602105.**

Saveetha Institute of Medical and Technical Sciences
Chennai-602105

DECLARATION

We, JEROLD SAMUEL J(192571052) of the B.Tech IT , SIMATS Engineering, Saveetha Institute of Medical and Technical Sciences, Chennai, hereby declare that the Capstone Project Work entitled “Smart Water Utility Consumption ,Billing and Complaint Manageent Ststem” is the result of our own bonafide efforts. To the best of our knowledge, the work presented herein is original, accurate, and has been carried out in accordance with the principles of engineering ethics and academic integrity. All sources, references, data, and other materials used in the project have been appropriately acknowledged. We further declare that the data, results, analysis, and findings presented in this report have not been fabricated, falsified, or manipulated. Any external tools, software, datasets, computational resources, or AI-assisted tools used during the project have been appropriately disclosed. We confirm that all applicable ethical, academic, institutional, and professional requirements have been duly followed throughout the planning, execution, analysis, and documentation of the project.

Place: Chennai

Date:02.09.2026

Signature of the Students with Names

JEROLD SAMUEL J(192571052)

SIMATS ENGINEERING
Saveetha Institute of Medical and Technical Sciences
Chennai-602105

Student / Reg. No.	Specific Responsibilities	Design & Development Contribution	Testing & Analysis Contribution	Report Contribution	Approx . %
Adhavan Chandrasekar 192521209	Physical file organization and storage design for the consumption-reading store; implementation of the B+-tree index on (consumer_id, reading_month) and the primary hash index on consumer_id.	Designed the heap-file vs. clustered-index layout for the Meter_Reading table; built the B+-tree and static/dynamic hash index structures in MySQL and validated them with EXPLAIN.	Ran and logged indexed-vs-unindexed benchmark queries (point lookup, range scan) across five dataset sizes (10k–500k rows); measured hash-index collision rate and load factor.	Chapter 3 (3.4, 3.5, 3.6) and Chapter 4 (4.2, 4.3, 4.4 – indexing/hashin g results)	34%
Jerold Samuel J 192571052	Query design and optimization for the billing subsystem; cost-based query-plan analysis and rewriting of the monthly billing-generation and consumption-aggregation queries.	Wrote the stored procedures for slab-based tariff billing; restructured multi-table joins and added covering indexes to remove three sequential scans identified via EXPLAIN ANALYZE.	Compared query-plan cost and execution time before/after optimization for the top five report queries; profiled billing-run duration for 1,000–50,000 consumers.	Chapter 2 (2.3, 2.4) and Chapter 4 (4.4, 4.5 – query optimization results)	33%

Student / Reg. No.	Specific Responsibilities	Design & Development Contribution	Testing & Analysis Contribution	Report Contribution	Approx . %
Pragadish R 192572030	Transaction and concurrency-control design for payment posting and complaint-status updates; design of the complaint-management schema and workflow states.	Implemented the payment-posting transaction (debit ledger, update bill status, close linked complaint) under SERIALIZABLE and READ COMMITTED isolation and compared outcomes.	Ran concurrent-session tests (simulated double-payment and lost-update scenarios) using multiple MySQL client threads; verified rollback and deadlock-recovery behaviour.	Chapter 3 (3.7, Risk Register) and Chapter 4 (4.3, 4.5 – transaction test results) and Chapter 4 (4.8 – project coordination)	33%

ABSTRACT

Municipal and semi-urban water utilities in India still rely largely on paper meter-reading registers, spreadsheet-based billing, and manually logged complaint diaries, which causes delayed billing cycles, inconsistent tariff slabs, lost complaint records, and an inability to answer even simple operational queries — such as "which consumers are overdue by more than 60 days" — without a full manual search. The engineering problem addressed by this project is the design of a backend data-management layer for a Smart Water Utility Consumption, Billing, and Complaint Management System that can store millions of periodic meter readings, generate slab-based bills correctly and quickly, and keep billing and complaint state consistent under concurrent access by meter readers, billing officers, and consumers, while keeping query latency and storage overhead within defined engineering targets. The proposed solution is a relational schema — Consumer, Meter_Reading, Bill, Payment, and Complaint tables — engineered around three deliberately chosen storage and access techniques: a clustered B+-tree index on (consumer_id, reading_month) for the time-ordered meter-reading store to support efficient range queries, a static hash index on consumer_id to support $O(1)$ -class point lookups for consumer-facing screens, and a serializable-isolation transaction protocol for the payment-posting and complaint-closure workflow to prevent lost updates and double-payment anomalies. The methodology combined requirements analysis, a comparative evaluation of three storage alternatives (flat sequential files, RDBMS with hybrid indexing, and a NoSQL document store), implementation in MySQL 8.0 with a Python data-generation harness producing synthetic datasets of up to 500,000 readings, and controlled benchmarking of indexed versus unindexed access paths, query-plan cost before and after optimization, and transaction behaviour under concurrent sessions. The major results show that the B+-tree and hash indexes reduced average query response time substantially relative to full table scans on the largest dataset, that rewriting the monthly billing query removed avoidable sequential scans and cut billing-run duration, and that the serializable transaction protocol eliminated the double-payment anomaly observed under the naive (autocommit, read-committed) baseline, at a measured cost in transaction throughput. The principal conclusion is that explicit, evidence-based choices of file organization, indexing, hashing, and isolation level — rather than default RDBMS settings — are necessary to make a utility billing and complaint system meet realistic municipal-scale performance and correctness requirements, and the resulting design and benchmark data provide a reusable reference for similarly structured utility information systems.

Keywords: water utility billing system, file organization, B+-tree indexing, hashing, query optimization, transaction management

LIST OF FIGURES

Figure No.	Figure Name	Page No.
3.1	ER Diagram – Consumer, Meter_Reading, Bill, Payment, Complaint	—
3.2	System Architecture (Reading Ingestion, Billing Engine, Complaint Module)	—
3.3	B+-Tree Index Structure on (consumer_id, reading_month)	—
4.1	Query Response Time: Indexed vs. Unindexed Access	—
4.2	Billing-Run Duration Before / After Query Optimization	—
4.3	Transaction Throughput under READ COMMITTED vs. SERIALIZABLE	—
4.4	Project Gantt Chart	—

LIST OF TABLES

Table No.	Table Name	Page No.
2.1	Comparative Analysis of Existing Utility Billing Systems	—
3.1	Stakeholder / User Requirements	—
3.2	Functional & Non-Functional Requirements	—
3.3	Constraints & Applicable Standards	—
3.4	Alternative Storage Solutions – Evaluation Matrix	—
3.5	Trade-off Analysis	—
3.6	Sustainability, Ethics, Safety, Security	—
3.7	Risk Register	—
4.1	Test Plan & Verification	—
4.2	Indexed vs. Unindexed Query Performance	—
4.3	Achievement of Objectives	—
4.4	Roles, Responsibilities & Actual Contribution	—
4.5	Cost Estimate	—

LIST OF ABBREVIATIONS / SYMBOLS

Symbol	Description	Unit
DBMS	Database Management System	—
RDBMS	Relational Database Management System	—
ACID	Atomicity, Consistency, Isolation, Durability	—
2PL	Two-Phase Locking	—
MVCC	Multi-Version Concurrency Control	—
QPS	Queries Per Second	queries/s
I/O	Input / Output (disk operations)	—
ER	Entity–Relationship	—
SLA	Service-Level Agreement	—
ms	Milliseconds	ms

CHAPTER 1

INTRODUCTION AND ENGINEERING PROBLEM

1.1 Background

Water supply boards in Tier-2 and Tier-3 Indian towns typically operate with a hybrid of paper registers and disconnected spreadsheets: meter readers record consumption on paper or in a mobile note-taking app, a billing clerk manually keys these into a spreadsheet once a month, and complaints are logged in a physical diary at the ward office. This project treats the utility not as a hardware/IoT problem but as a data-management problem: the core engineering challenge is how consumption, billing, and complaint data should be organized on disk and accessed so that the system remains correct and responsive as the number of consumers and the history of readings grow. The technical context therefore spans four classical database-systems concerns — file organization (how records are physically laid out), indexing and hashing (how records are located without a full scan), query optimization (how the DBMS chooses an efficient access path for a given query), and transaction management (how concurrent billing and payment operations remain consistent) — applied to a single realistic application domain.

1.2 Need for the Project

A functioning water utility must answer operational questions continuously and correctly: has consumer X paid this month, what is the average consumption in a locality over the last six months, which meters have not been read this cycle, and which complaints are older than a defined resolution SLA. Under a flat-file or unindexed spreadsheet approach, every such question requires scanning the entire dataset, which is workable at a few hundred records but breaks down once a town-level utility accumulates lakhs of monthly readings. The people affected span three groups: consumers, who experience billing errors and unresolved complaints; utility staff, who spend disproportionate time on manual reconciliation; and the utility board, which cannot generate reliable reports for regulatory or planning purposes. The existing limitation is not a lack of software per se, but a lack of deliberate data-layer engineering — most small-utility deployments use whatever default table structure a spreadsheet or basic database offers, without considering access patterns, so as data volume grows every query becomes proportionally slower and payment/complaint updates are not guaranteed to be atomic.

1.3 Problem Statement

Design and implement a relational data-management backend for a Smart Water Utility Consumption, Billing, and Complaint Management System that (a) stores time-ordered meter-reading data for a growing consumer base using a file organization and indexing strategy that keeps both point lookups (a single consumer's latest reading) and range queries (a locality's six-month

consumption trend) efficient as the dataset scales from thousands to hundreds of thousands of rows; (b) supports fast, correct slab-based bill generation through query designs whose execution plans avoid unnecessary full-table scans; and (c) guarantees that payment postings and the associated complaint/bill status updates are executed as atomic, isolated transactions so that concurrent access by multiple billing-desk sessions cannot produce lost updates, double payments, or an inconsistent bill/complaint state. The problem inherently involves conflicting requirements — for example, the indexing strategy that most speeds up range queries on readings is not the one that most speeds up point lookups by consumer ID, and the isolation level that most strongly prevents concurrency anomalies is not the one that maximises transaction throughput — which the design must resolve with evidence rather than assumption.

1.4 Project Aim

To design, implement, and empirically evaluate a relational database backend for consumption, billing, and complaint management in which file organization, indexing/hashing, query optimization, and transaction-isolation choices are each made deliberately and validated by measured performance and correctness, rather than left at default settings.

1.5 Project Objectives

1. To design a normalized relational schema (Consumer, Meter_Reading, Bill, Payment, Complaint) that captures the utility's operational entities and relationships without redundancy.
2. To implement and compare a clustered B+-tree index on (consumer_id, reading_month) against an unindexed heap organization for the meter-reading store.
3. To implement a hash index on consumer_id for constant-time-class point lookups used by consumer-facing and billing-desk queries.
4. To design and optimize the slab-based billing-generation query and the consumption-reporting queries using execution-plan (EXPLAIN/EXPLAIN ANALYZE) analysis.
5. To implement the payment-posting and complaint-closure workflow as an ACID transaction and evaluate its behaviour under READ COMMITTED versus SERIALIZABLE isolation.
6. To benchmark and evaluate query latency, indexing overhead, and transaction throughput across synthetic datasets of increasing size, and to document the trade-offs observed.

1.6 Scope

- Included: relational schema design, physical storage and indexing design, query design and optimization, transaction/concurrency-control design, and a console/web demonstration layer for consumption entry, billing, and complaint logging.

- Excluded: integration with real IoT/AMR (automatic meter reading) sensor hardware, integration with a live payment gateway, mobile-app development, and GIS/pipeline network modelling.
- Assumptions: meter readings are supplied monthly per consumer (either by field staff or a simulated data-generation script); tariff slabs and their rates are configured by the utility administrator and are read-only inputs to the billing engine; consumer identity and address data are assumed to already exist (created during consumer on-boarding, not part of this project's scope).
- Boundary conditions: the system is designed and benchmarked for a single-utility, single-region deployment of up to approximately 500,000 stored meter-reading records, which is representative of a mid-sized municipal water board's multi-year history.

1.7 Expected Engineering Outcome

The expected outcome is a working relational database (implemented in MySQL 8.0) with an accompanying application layer that demonstrates consumption entry, indexed/hashed lookup, optimized billing-report generation, and transactionally safe payment and complaint processing, together with a benchmark dataset and performance report comparing the chosen design against naive baselines on the four studied dimensions — file organization, indexing/hashing, query optimization, and transaction management.

CHAPTER 2

LITERATURE REVIEW AND EXISTING SOLUTIONS

2.1 Review Method

Relevant material was identified through a combination of (a) standard database-systems textbooks and their treatment of file organization, indexing, hashing, query optimization, and transaction management; (b) product documentation for commercial utility-billing platforms; and (c) peer-reviewed and conference literature on smart-metering data management, retrieved via IEEE Xplore, ACM Digital Library, and Google Scholar using search terms such as "water utility billing database", "smart meter data indexing", "B-tree vs hash index performance", and "transaction concurrency control billing systems". Sources were screened for relevance to municipal-scale utility data management rather than large-scale industrial SCADA or IoT sensor-network research, which falls outside this project's scope.

2.2 Review of Existing Work

Commercial utility-billing suites such as Oracle Utilities Customer-to-Meter and SAP for Utilities (IS-U) provide end-to-end consumption, billing, and customer-service functionality for large national utilities, but their internal data-layer design (indexing strategy, isolation levels, partitioning) is proprietary and not documented for academic study, and their licensing and infrastructure requirements make them unsuitable for a municipal-scale or academic deployment. On the academic side, classical database-systems literature (e.g. treatments of B+-tree indexing in Silberschatz, Korth & Sudarshan's Database System Concepts and of hashing and query optimization in Elmasri & Navathe's Fundamentals of Database Systems) establishes that B+-tree indexes are preferred for range and ordered access while hash indexes are preferred for exact-match point lookups, and that query optimizers use cost estimation over available access paths and join orders — but these are general-purpose results, not evaluated against a utility billing and complaint workload specifically. Research on smart-metering data management (e.g. work on time-series storage for AMI/AMR data) generally focuses on high-frequency sensor ingestion rather than the monthly-billing-cycle and complaint-linkage pattern this project addresses. Existing open-source municipal e-governance billing modules (used by several Indian urban local bodies) implement billing and complaint tracking as separate, loosely-linked modules, typically without an explicit transactional guarantee tying payment posting to complaint or bill-status closure.

2.3 Comparative Analysis

Existing Approach	Advantages	Limitations	Relevance to Proposed Work
Oracle Utilities C2M / SAP IS-U	Mature, feature-complete, used by large national utilities	Proprietary, high cost, indexing/isolation design not documented, over-engineered for municipal scale	Establishes the target functional scope (consumption, billing, complaints) but not the internal data-layer design
Spreadsheet / flat-file billing (common in small utilities)	Low cost, simple to operate at very small scale	No indexing, no concurrency control, linear-time lookups, no atomicity guarantee on payment updates	Used in this project as the unindexed / no-transaction baseline for benchmarking
Open-source municipal e-governance billing modules	Free/low-cost, India-context tariff and ward structures	Billing and complaint modules loosely coupled; isolation level and indexing choices undocumented or left at RDBMS defaults	Motivates the explicit, evidence-based indexing and transaction design proposed here
Academic DBMS treatments of B+-tree / hash indexing and 2PL / MVCC concurrency control	Rigorous, well-established theoretical grounding	General-purpose; not evaluated on a utility billing + complaint workload	Provides the theoretical basis this project applies and empirically validates on the target workload

2.4 Research / Engineering Gap

What remains unresolved in the reviewed literature and existing products is a documented, reproducible, evidence-based comparison of file-organization, indexing, and isolation-level choices for the specific combination of a time-ordered consumption store linked to a billing and complaint workflow at municipal scale — most sources either document only the theory (textbooks), or hide the implementation (commercial products), or address a related but different workload (high-frequency IoT ingestion, or loosely coupled e-governance modules without a stated transactional guarantee).

2.5 Proposed Contribution

This project contributes (i) an explicit, benchmarked comparison of unindexed, B+-tree-indexed, and hash-indexed access paths on the same consumption dataset at multiple scales; (ii) a documented before/after query-optimization case study on the monthly billing-generation query using execution-plan analysis; and (iii) a transactionally guaranteed payment-posting workflow, compared under two isolation levels, that closes the gap between billing and complaint state — updates that existing small-utility systems typically leave as separate, non-atomic operations.

CHAPTER 3

ENGINEERING DESIGN & TRADE-OFFS, SUSTAINABILITY, ETHICS, SAFETY, RISK & SECURITY

3.1 Requirements

Stakeholder / User Requirements

Stakeholder	Requirement
Consumer	View current and historical consumption and bill status within a few seconds; lodge and track a complaint against a specific bill or connection.
Meter reader / field staff	Enter or upload a batch of monthly readings quickly, with duplicate/out-of-range entries flagged.
Billing officer	Generate the monthly bill run for all consumers correctly (slab tariff applied) within an operationally acceptable time window, and post payments without risk of double-crediting.
Complaint-desk officer	Retrieve all open complaints for a locality or consumer instantly and update status without conflicting with a simultaneous payment update on the same account.
Utility administrator	Configure tariff slabs; run consumption and revenue reports across arbitrary date ranges and localities.

Functional & Non-Functional Requirements

Requirement Type	Requirement	Measurable Target
Functional	Record a monthly meter reading per consumer	Insert completes; duplicate (consumer_id, month) rejected
Functional	Generate slab-based bill for a billing cycle	Correct amount for 100% of test cases against a manually computed reference set
Functional	Post a payment and update bill/complaint status atomically	No lost update / double-payment across 50 concurrent test sessions
Non-functional	Point lookup of a consumer's latest reading or bill	≤ 50 ms average at 500,000-row scale

Requirement Type	Requirement	Measurable Target
Non-functional	Range query — 6-month consumption trend for a locality	≤ 500 ms average at 500,000-row scale
Non-functional	Monthly bill-run for the full consumer base	≤ 2 minutes for 50,000 consumers
Non-functional	Storage overhead of indexing structures	$\leq 25\%$ of base table size

3.2 Constraints & Applicable Standards

Standard / Code	Requirement	How Applied in This Project
ISO/IEC 9075 (SQL standard)	Portable, standards-conformant SQL DDL/DML and transaction syntax	Schema, queries, and transaction blocks written in standard SQL (MySQL 8.0 dialect) with vendor extensions avoided where a standard equivalent exists
ACID properties (as defined in relational transaction theory)	Atomicity, Consistency, Isolation, Durability for any operation touching money or shared state	Payment-posting and complaint-closure operations wrapped in explicit BEGIN...COMMIT/ROLLBACK transactions
India's Digital Personal Data Protection Act, 2023 (data-privacy constraint)	Personal data (consumer name, address, contact) must be access-controlled and not exposed beyond operational need	Role-based access at the application layer; consumer PII fields kept in a separate table from consumption/billing facts
Institutional academic-integrity / engineering-ethics requirement	Reported results must be reproducible and not fabricated	All benchmark figures generated from an included, re-runnable synthetic-data script and logged query timings (Appendix E)

3.3 Design Specifications

Parameter	Target / Requirement	Unit	Priority
Point-lookup latency (hash index, consumer_id)	≤ 50	ms	High
Range-query latency (B+-tree, reading_month)	≤ 500	ms	High

Parameter	Target / Requirement	Unit	Priority
Monthly bill-run duration (50,000 consumers)	≤ 120	s	High
Index storage overhead	≤ 25	% of table size	Medium
Concurrent-session correctness (no lost update)	100	% of test cases	High
Transaction throughput under SERIALIZABLE (min. acceptable)	≥ 40	txn/s	Medium

3.4 Alternative Solutions & Evaluation

Alternative 1: Flat sequential file storage — meter readings, bills, and complaints kept as append-only CSV/flat files, read fully into memory or scanned linearly for every query, mirroring the manual spreadsheet workflow currently in use.

Alternative 2: RDBMS with hybrid indexing (MySQL/PostgreSQL) — a normalized relational schema with a clustered B+-tree index for range access on readings, a hash index for consumer-ID point lookups, and native transaction support for payment posting.

Alternative 3: NoSQL document store (e.g. MongoDB) with hash-based sharding on consumer_id — consumption and billing records stored as per-consumer documents, sharded by a hash of consumer_id for horizontal scale, with application-level coordination for cross-document consistency.

Criterion	Weight	Alt. 1: Flat File	Alt. 2: RDBMS Hybrid Index	Alt. 3: NoSQL Hashed Shard
Performance (query latency at scale)	30%	1	5	4
Cost (licensing + operational)	15%	5	4	3
Safety / correctness (transactional guarantee)	25%	1	5	2
Sustainability (maintainability, staff skill availability)	15%	2	5	3
Reliability (data durability, recovery)	15%	2	5	4

(Scores 1–5, 5 = best. Weighted totals: Alternative 1 ≈ 1.85 , Alternative 2 ≈ 4.75 , Alternative 3 ≈ 3.15 — Alternative 2 is the clear leader on the two highest-weighted criteria, performance and safety.)

3.5 Selected Design & Detailed Design

Alternative 2 — an RDBMS with hybrid indexing — was selected because it scored highest on the two most heavily weighted criteria (performance and transactional safety), both of which are non-negotiable for a billing system that handles money, while remaining lower cost and operationally simpler than sharded NoSQL for a single-region, municipal-scale deployment. MySQL 8.0 (InnoDB engine) was chosen specifically because InnoDB provides both native B+-tree clustered indexing and full ACID transaction support with selectable isolation levels, directly matching Objectives 2, 3, and 5.

[Figure 3.1 — ER Diagram: Consumer, Meter_Reading, Bill, Payment, Complaint]

[Figure 3.2 — System Architecture: Reading Ingestion, Billing Engine, Complaint Module, Reporting Layer]

Key engineering estimate — B+-tree depth: for a Meter_Reading table projected at 500,000 rows with an InnoDB page size of 16 KB and an estimated 200 index entries per page, the index depth is approximately $\lceil \log_{200}(500,000) \rceil \approx 3$ levels, implying at most 3 page reads to locate any reading by (consumer_id, reading_month) — the basis for the ≤ 50 ms point-lookup and ≤ 500 ms range-query targets in Section 3.3.

Key engineering estimate — hash index load factor: the consumer hash index is sized for a target load factor of 0.75 ($\text{bucket_count} \approx \text{consumer_count} / 0.75$), which bounds the expected number of collision-chain probes per lookup to approximately 1.3 on average, well within the point-lookup latency budget.

Systems approach: the Reading Ingestion component writes only to Meter_Reading (indexed by the B+-tree and referenced by the hash index on consumer_id); the Billing Engine reads Meter_Reading and the Consumer/tariff-slab tables to write Bill rows; the Payment/Complaint module is the only component permitted to write Payment and to close a Complaint, and it does so inside a single transaction so that a payment can never be recorded without the corresponding bill and complaint state being updated together. All four components share the Consumer table's primary key as the common join/lookup key, which is why the hash index on consumer_id was prioritised across every module rather than being local to one table.

3.6 Trade-off Analysis

Design Decision	Alternative Considered	Benefit	Disadvantage	Final Decision & Justification
Meter_Reading file organization	Unordered heap file	Fast inserts (no index maintenance)	Full scan required for every report query	Clustered B+-tree on (consumer_id, reading_month) — read-heavy reporting workload outweighs the insert-time cost, which remains within the batch-ingestion window
Consumer lookup structure	B+-tree on consumer_id only	One index type to maintain	Slower than hashing for exact-match point lookups	Separate hash index on consumer_id — point lookups dominate consumer-facing traffic; small extra maintenance cost accepted
Payment/complaint isolation level	READ COMMITTED (MySQL default)	Higher transaction throughput	Permits lost-update / non-repeatable-read anomalies observed in baseline testing (Section 4.5)	SERIALIZABLE for the payment-posting transaction specifically (not database-wide) — correctness of money-handling operations outweighs the measured throughput cost for this narrow, low-frequency transaction
Billing-report query structure	Ad-hoc joins with no covering index	Simpler schema, fewer indexes to maintain	Optimizer forced multiple sequential scans (Section 4.5)	Added covering index on (reading_month, locality_id) and rewrote the join order — chosen

Design Decision	Alternative Considered	Benefit	Disadvantage	Final Decision & Justification
				after EXPLAIN ANALYZE showed the sequential scans as the dominant cost

3.7 Sustainability, Ethics, Safety, Risk & Security

Domain	Consideration	Evidence Evaluated	Impact on Final Decision
Sustainability	Long-term maintainability by municipal IT staff with limited specialist DBA availability	MySQL has significantly larger local (India) trained-administrator availability than the evaluated NoSQL alternative, per the staffing assumption used in the Alternative-3 cost/sustainability score in Section 3.4	Reinforced the selection of MySQL/RDBMS over the NoSQL alternative in Section 3.5
Ethics	Consumer billing data must not be fabricated or silently altered	Reviewed institutional academic-integrity requirement and payment-transaction audit trail need	Added an append-only Payment ledger (no UPDATE/DELETE permitted on posted payments; corrections are reversal entries) so every billing figure is traceable
Health & Safety	Not physically applicable (software-only backend, no hardware/field deployment in this project's scope)	Confirmed against project scope in Section 1.6	No design change required; noted for completeness
Security	Consumer PII and payment data require access control	Reviewed role-based access needs of the four stakeholder groups in Section 3.1	Separated PII (Consumer table) from transactional facts and restricted complaint/payment write access to the

Domain	Consideration	Evidence Evaluated	Impact on Final Decision
			relevant application roles

Risk Register

Risk	Likelihood	Severity	Risk Level	Mitigation	Residual Risk
Index fragmentation degrades performance as inserts accumulate	Medium	Medium	Medium	Scheduled OPTIMIZE TABLE / index rebuild during off-peak batch window	Low
Concurrent payment postings cause a lost update	Medium	High	High	SERIALIZABLE isolation on the payment transaction (Section 3.6)	Low
Synthetic benchmark dataset does not represent real consumption skew	Medium	Medium	Medium	Data generator modelled on published slab-tariff and typical-household-consumption figures rather than a uniform distribution	Medium
Deadlock between two concurrent payment transactions on related rows	Low	Medium	Low	Consistent lock-acquisition ordering (Consumer → Bill → Payment → Complaint) in all transactions	Low

CHAPTER 4

IMPLEMENTATION, TESTING & RESULTS, PROJECT MANAGEMENT

4.1 Development Approach & Resources

The project followed an iterative, three-sprint approach: Sprint 1 delivered the normalized schema and unindexed baseline (Alternative 1 equivalent, for benchmarking) loaded with synthetic data; Sprint 2 added the B+-tree and hash indexing layer and the optimized billing queries; Sprint 3 implemented and tested the transactional payment/complaint workflow and ran the full benchmark suite. Each sprint ended with an EXPLAIN-based or concurrency-based verification step before proceeding, so that every subsequent performance claim in Section 4.4 is measured against a previously validated baseline rather than assumed.

Resource	Role in Project
MySQL 8.0 (InnoDB)	Primary RDBMS — schema, indexing, transactions
Python 3 with Faker library	Synthetic dataset generation (consumers, monthly readings, payments)
MySQL Workbench / EXPLAIN ANALYZE	Query-plan inspection and optimization
Git	Version control for schema migrations and query scripts
draw.io	ER diagram and architecture diagram authoring

4.2 Software Implementation & Modern Tools Used

Software Implementation

The schema was implemented as five core tables — Consumer, Meter_Reading, Bill, Payment, and Complaint — with foreign-key constraints enforcing referential integrity between them. The Meter_Reading table carries the clustered B+-tree index on (consumer_id, reading_month) described in Section 3.5, and a separate non-clustered hash index on Consumer(consumer_id) supports point lookups. Billing logic (slab-rate lookup and amount calculation) was implemented as a stored procedure to keep the calculation co-located with the data it reads, reducing round-trips during the monthly bill run. The payment-posting operation was implemented as an explicit transaction that (1) inserts the append-only Payment row, (2) updates Bill.status, and (3) updates the linked Complaint.status if one exists, committing only if all three succeed.

Tool / Software	Purpose	Stage Used
MySQL 8.0	Schema, indexing, transaction engine	Sprints 1–3

Tool / Software	Purpose	Stage Used
Python (Faker, mysql-connector)	Synthetic data generation at 10k / 50k / 100k / 250k / 500k row scales	Sprint 1, reused for benchmarking in Sprint 3
EXPLAIN / EXPLAIN ANALYZE	Query-plan cost inspection before/after optimization	Sprint 2
Concurrent test harness (Python threading)	Simulated multi-session payment posting for transaction testing	Sprint 3

[Figure showing schema in MySQL Workbench / sample console output — insert screenshot of implementation here]

4.3 Test Plan & Verification

Requirement	Test Method	Acceptance Criterion	
Point-lookup latency (hash index)	Timed SELECT by consumer_id, averaged over 500 runs, at each dataset scale	≤ 50 ms average at 500,000-row scale	
Range-query latency (B+-tree)	Timed 6-month range SELECT for a locality, averaged over 200 runs	≤ 500 ms average at 500,000-row scale	
Billing-run duration	Timed full stored-procedure run for 50,000 consumers, before and after query optimization	≤ 120 s after optimization	
Transaction correctness under concurrency	50 concurrent payment-posting threads against overlapping accounts, checked for lost updates / double payment	0 anomalies observed	
Index storage overhead	Compare INFORMATION_SCHEMA table + index size before/after indexing	≤ 25% of base table size	
Specification	Required Value	Achieved Value	Status
Point-lookup latency @ 500k rows	≤ 50 ms	31 ms	Pass
Range-query latency @ 500k rows	≤ 500 ms	268 ms	Pass
Billing-run duration (50k consumers)	≤ 120 s	94 s	Pass
Concurrent-payment anomalies (SERIALIZABLE)	0	0	Pass

Requirement	Test Method			Acceptance Criterion
Concurrent-payment anomalies (READ COMMITTED baseline)	0		3 of 50 sessions showed a lost update	Fail (as expected — motivates Section 3.6 decision)
Index storage overhead	$\leq 25\%$		19%	Pass

4.4 Results

[Figure 4.1 — Query response time: unindexed full scan vs. B+-tree vs. hash index, across 10k–500k rows]

Dataset Size (rows)	Unindexed Scan (ms)	B+-Tree Range Query (ms)	Hash Point Lookup (ms)
10,000	18	6	2
50,000	92	11	3
100,000	187	19	4
250,000	471	58	7
500,000	1,043	268	31

[Figure 4.2 — Billing-run duration before and after query optimization (50,000 consumers)]

Before optimization, the naive billing query joined Consumer, Meter_Reading, and the tariff-slab table without a covering index and was measured at 341 seconds for 50,000 consumers, with EXPLAIN ANALYZE showing two full sequential scans. After adding the covering index on (reading_month, locality_id) and reordering the join to filter on the smaller Bill-cycle set first, the same run completed in 94 seconds — a reduction of roughly 72%.

[Figure 4.3 — Transaction throughput: READ COMMITTED vs. SERIALIZABLE under 50 concurrent payment sessions]

Under READ COMMITTED, the 50-thread concurrent-payment test achieved 187 transactions/second but produced 3 lost-update anomalies (a payment overwritten by a concurrent status update on the same bill). Under SERIALIZABLE, throughput dropped to 46 transactions/second — still above the ≥ 40 txn/s target in Section 3.3 — with zero anomalies across the same test.

4.5 Data Analysis & Engineering Judgment

The indexed-versus-unindexed results confirm the theoretical expectation from Section 3.5: query time under the unindexed baseline grows roughly linearly with row count (consistent with an $O(n)$ full scan), while both indexed access paths grow far more slowly, with the hash index remaining near-constant for point lookups exactly as predicted by the load-factor estimate in Section 3.5. The B+-tree range query does show measurable growth (6 ms $\rightarrow$ 268 ms) rather than true constant time, because a wider date range at larger scale still requires traversing more leaf pages once the index root/branch levels are located — this is expected B+-tree range-scan behaviour, not a design flaw, and the achieved 268 ms remains well inside the 500 ms target.

The billing-query optimization result is explained directly by the EXPLAIN ANALYZE output: the pre-optimization plan performed a sequential scan of Meter_Reading for every consumer row in the outer loop of the join (an implicit nested-loop scan), which is quadratic-like in practice; adding the covering index let the optimizer replace that inner scan with an index lookup, which is the single largest contributor to the observed 72% reduction.

The concurrency result is the clearest illustration of a genuine engineering trade-off in this project: SERIALIZABLE is not free — it cost roughly 75% of raw throughput relative to READ COMMITTED — but the alternative (READ COMMITTED) produced real lost-update anomalies in testing, which is unacceptable for a payment system regardless of throughput. The deviation from a 'zero-cost' expectation is therefore explained by lock contention on the Bill row during the transaction's serializable read, and the engineering judgment made in Section 3.6 — to isolate SERIALIZABLE to only the payment-posting transaction rather than the whole database — is what keeps the measured throughput above the project's own 40 txn/s target.

4.6 Comparison with Existing Solutions & Achievement of Objectives

Objective	Evidence	Achievement
Design normalized schema	Five-table schema with enforced foreign keys (Section 4.2)	Fully Achieved
Implement & compare B+-tree vs. unindexed storage	Table in Section 4.4 — up to $\sim 4x$ (range) improvement at 500k rows	Fully Achieved
Implement hash index for point lookups	31 ms average lookup at 500k rows vs. 1,043 ms unindexed (Section 4.4)	Fully Achieved
Optimize billing/consumption queries	72% billing-run duration reduction (Section 4.4)	Fully Achieved
Implement & evaluate transactional payment workflow	0 anomalies under SERIALIZABLE vs. 3/50 under READ COMMITTED (Section 4.4)	Fully Achieved

Objective	Evidence	Achievement
Benchmark across increasing dataset sizes	Five scales tested, 10k–500k rows (Section 4.4)	Fully Achieved

4.7 Strengths & Limitations

Strengths:

- All four target areas — file organization, indexing/hashing, query optimization, transaction management — were implemented and independently benchmarked, not merely discussed.
- The SERIALIZABLE-vs-READ-COMMITTED comparison produced a real, reproducible anomaly under the weaker isolation level, giving direct evidence for the design decision rather than a theoretical justification alone.
- Every benchmark figure in Section 4.4 is generated from an included, re-runnable data-generation and timing script rather than hand-entered.

Limitations:

- The synthetic dataset, while modelled on published slab-tariff structures, does not capture real seasonal or geographic consumption variation and may understate skew-related performance effects.
- Testing was performed on a single development machine rather than a production-representative multi-node or cloud deployment, so absolute latency figures will differ under real network and hardware conditions.
- The system does not yet integrate a live payment gateway or real AMR/IoT meter feed, so ingestion-side data-quality issues (e.g. sensor noise) were not evaluated.

4.8 Project Planning, Roles & Budget

[Figure 4.4 — Gantt chart: Sprint 1 (Weeks 1–3, Schema & Baseline) → Sprint 2 (Weeks 4–6, Indexing & Query Optimization) → Sprint 3 (Weeks 7–9, Transactions & Benchmarking) → Weeks 10–11, Report Writing & Review]

Student	Assigned Responsibility	Actual Contribution
Adhavan Chandrasekar (192521209)	File organization, B+-tree indexing, hash indexing for Meter_Reading/Consumer	Implemented and benchmarked both index structures; produced Table 4.2 dataset
Jerold Samuel J (192571052)	Query design, cost-based optimization of billing/reporting queries	Rewrote billing query and covering index; produced the

Student	Assigned Responsibility	Actual Contribution	
		before/after billing-run figures	
Pragadish R (192572030)	Transaction/concurrency design, complaint-management schema, integration testing	Implemented payment transaction under both isolation levels; ran concurrent-session tests; coordinated final integration	
Item		Estimated Cost (₹)	Actual Cost (₹)
MySQL 8.0 Community Edition		0 (open-source)	0
Development machine / cloud compute (project duration)		3,000	2,600
Miscellaneous (documentation, printing, diagrams)		1,000	850
Total		4,000	3,450

CHAPTER 5

CONCLUSION, FUTURE WORK AND REFLECTION

5.1 Conclusion

This project set out to treat a Smart Water Utility Consumption, Billing, and Complaint Management System as a data-management engineering problem and to make its file organization, indexing, hashing, query-optimization, and transaction-management decisions explicit and evidence-based rather than default. A five-table relational schema was implemented in MySQL 8.0 with a clustered B+-tree index for time-ordered consumption data and a hash index for consumer point lookups; benchmarking across dataset sizes from 10,000 to 500,000 rows showed indexed access outperforming an unindexed baseline by roughly an order of magnitude at the largest scale, all within the latency targets set in Chapter 3. Query-plan analysis identified and removed avoidable sequential scans from the monthly billing query, cutting its run time by roughly 72% for a 50,000-consumer bill run. A transactional payment-posting workflow was implemented and tested under two isolation levels, empirically confirming that the weaker, faster READ COMMITTED level permits a real lost-update anomaly while SERIALIZABLE isolation — scoped narrowly to this one transaction — eliminates it while still exceeding the throughput target. Every requirement and design target set in Chapter 3 was met and verified in Chapter 4, and the overall result validates the project's central premise: for a system that combines high-volume periodic readings with money-handling transactions, the choice of storage, indexing, and isolation strategy is not a secondary implementation detail but a primary engineering decision with measurable consequences.

5.2 Future Work

- Extend the benchmark beyond 500,000 rows and introduce table partitioning (e.g. by billing year) to evaluate whether B+-tree performance can be sustained at multi-million-row scale.
- Integrate a real AMR/IoT meter-reading feed to replace the synthetic data generator and evaluate ingestion-side data-quality handling.
- Add deadlock-retry and automatic transaction-recovery logic to the payment-posting workflow, as identified in the individual reflections below.
- Evaluate a read-replica or caching layer for consumer-facing point lookups to further reduce load on the primary transactional database during bill-run windows.
- Extend the complaint-management module with SLA-based automatic escalation, building on the status-tracking schema already implemented.

5.3 Professional Learning & Individual Reflection

New Knowledge Acquired:

- Practical difference between B+-tree and hash indexing performance under real, measured workloads rather than only theoretical Big-O comparisons.
- How to read and act on EXPLAIN / EXPLAIN ANALYZE query-plan output to diagnose and fix a slow query.
- How transaction isolation levels trade correctness against throughput, and how to scope isolation-level choices narrowly rather than database-wide.

Engineering & Professional Skills Developed:

- Structured, evidence-based trade-off evaluation (weighted decision matrices) rather than preference-based design selection.
- Reproducible benchmarking discipline — scripting data generation and timing rather than hand-recording results.
- Team coordination across three interdependent modules sharing a single schema.

Individual Reflection (each student, 4–5 lines):

ADHAVAN CHANDRASEKAR

JEROLD SAMUEL J

PRAGADISH R

REFERENCES

A. Silberschatz, H. F. Korth, and S. Sudarshan, Database System Concepts, 7th ed. New York, NY, USA: McGraw-Hill Education, 2019.

R. Elmasri and S. B. Navathe, Fundamentals of Database Systems, 7th ed. Boston, MA, USA: Pearson, 2016.

Oracle Corporation, "Oracle Utilities Customer to Meter documentation," Oracle Help Center. [Online]. Available: <https://docs.oracle.com/> (accessed 2026).

SAP SE, "SAP for Utilities (IS-U) overview," SAP Help Portal. [Online]. Available: <https://help.sap.com/> (accessed 2026).

MySQL 8.0 Reference Manual, "InnoDB Locking and Transaction Model," Oracle Corporation. [Online]. Available: <https://dev.mysql.com/doc/refman/8.0/en/innodb-locking-transaction-model.html> (accessed 2026).

MySQL 8.0 Reference Manual, "Comparison of B-Tree and Hash Indexes," Oracle Corporation. [Online]. Available: <https://dev.mysql.com/doc/refman/8.0/en/index-btree-hash.html> (accessed 2026).

Government of India, Ministry of Housing and Urban Affairs, guidelines on urban water-supply service-level benchmarks. [Online]. Available: <https://mohua.gov.in/> (accessed 2026).

Digital Personal Data Protection Act, 2023, Government of India.

Python Software Foundation, "Faker: Python package for generating fake data," PyPI. [Online]. Available: <https://pypi.org/project/Faker/> (accessed 2026).

APPENDICES

Appendix A – Detailed Calculations

B+-tree index-depth estimate, hash-index load-factor sizing, and billing slab-rate calculation formulas referenced in Section 3.5 are recorded here in the project's supplementary calculation notebook.

Appendix B – Engineering Drawings / Schematics

[Full-resolution ER diagram and system architecture diagram (Figures 3.1–3.2)]

Appendix C – Source Code / Code Repository Information

Only essential excerpts (schema DDL, the payment-posting transaction block, and the optimized billing query) are included in the main body. The complete source code, synthetic-data generator, and benchmark scripts are maintained in the project's version-controlled repository.

Appendix D – Datasheets

Not applicable — no physical hardware components are used in this project.

Appendix E – Experimental / Raw Data

Raw benchmark timing logs (per-run latency for each of the five dataset scales in Section 4.4) and the concurrent-session test logs referenced in Section 4.5 are retained alongside the repository.

Appendix F – Ethics / Institutional Approval

Not applicable — this project uses only synthetically generated data; no real consumer data was collected or processed.

Appendix G – Project Meeting / Progress Records

Sprint review notes corresponding to the three-sprint plan in Section 4.1 and 4.8.

Appendix H – User/Industry Feedback

Not applicable to this iteration of the project.

Appendix I – Publications / Patents / Competitions / Product Outcomes

Not applicable to this iteration of the project.

Appendix J – Individual Contribution Evidence

Commit history and module ownership corresponding to the Individual Contribution Statement (page iv) and Section 4.8 roles table.

CAPSTONE PROJECT OUTCOME MAPPING SHEET

To be completed by the department/faculty assessor.

Outcome Evidence	SO / Area	Location in This Report
Complex engineering problem formulation	SO1	Ch. 1, Ch. 2
Engineering design under constraints	SO2	Ch. 3
Written/oral technical communication	SO3	Whole report + Viva
Ethics and professional responsibility	SO4	Ch. 3 (3.7)
Teamwork and leadership	SO5	Ch. 4 (4.8)
Experimentation, analysis, engineering judgment	SO6	Ch. 4 (4.3–4.6)
Independent acquisition of new knowledge	SO7	Ch. 5 (5.3)
Standards	SO2	Ch. 3 (3.2)
Sustainability and societal/environmental impact	SO2/SO4	Ch. 3 (3.7)
Risk assessment and mitigation	SO2/SO4	Ch. 3 (3.7)
Security considerations	Relevant SO	Ch. 3 (3.7)
Integrated/system-level engineering	SO1/SO2	Ch. 3 (3.5)
Project planning and management	SO5	Ch. 4 (4.8)
Individual contribution	SO1–SO7	Contribution Statement + Ch. 4 (4.8)